Práctica 3: Tetris

Programación 2

Curso 2025-2026

Esta práctica consiste en implementar el clásico juego Tetris, con alguna simplificación. Los conceptos necesarios para desarrollar esta práctica se trabajan en todos los temas de teoría, aunque especialmente en el *Tema 5*.

Condiciones de entrega

- La fecha límite de entrega para esta práctica es el **viernes 22 de mayo**, hasta las **23:59**
- La práctica consta de varios ficheros: `Coordinate.cc`, `Coordinate.h`, `Piece.cc`, `Piece.h`, `Gameboard.cc`, `Gameboard.h`, `Game.cc`, `Game.h`. Todos ellos se deberán comprimir en un único fichero llamado `prog2p3.tgz` que se entregará a través del servidor de prácticas de la forma habitual. Para crear el fichero comprimido debes hacerlo de la siguiente manera (en una única línea):

Terminal

```
$ tar cvfz prog2p3.tgz Coordinate.cc Coordinate.h
            Piece.cc Piece.h Gameboard.cc Gameboard.h Game.cc Game.h
```

Código de honor

Si se detecta copia (total o parcial) en tu práctica, tendrás un **0** en la entrega y se informará a la dirección de la Escuela Politécnica Superior para que adopte medidas disciplinarias

Está bien discutir con tus compañeros posibles soluciones a las prácticas
Está bien apuntarte a una academia si sirve para obligarte a estudiar y hacer las prácticas

Está mal copiar código de otros compañeros o pedirle a ChatGPT que te haga la práctica
Está mal apuntarte a una academia para que te hagan las prácticas

Si necesitas ayuda acude a tu profesor/a
No copies

Normas generales

- Debes entregar la práctica exclusivamente a través del servidor de prácticas del Departamento de Lenguajes y Sistemas Informáticos (DLSI): https://pracdlsi.dlsi.ua.es
- Cuestiones que debes tener en cuenta al hacer la entrega:
  - El usuario y la contraseña para entregar prácticas son los mismos que utilizas en UACloud

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

    - Puedes entregar la práctica varias veces, pero sólo se corregirá la última entrega
    - No se admitirán entregas por otros medios, como el correo electrónico o UACloud
    - No se admitirán entregas fuera de plazo

- Tu práctica debe poder ser compilada sin errores con el compilador de C++ existente en la distribución de Linux de los laboratorios de prácticas
- Si tu práctica no se puede compilar, su calificación será 0
- Si tu práctica utiliza instrucciones que no se han visto en el temario de la asignatura, su calificación será 0
- En esta práctica el 100 % de la nota dependerá de la corrección automática. **No habrá revisión de estilo por parte de ChatGPT ni del profesor**. Aún así, se espera que sigas la guía de estilo de la asignatura en tu código, igual que has hecho en las dos prácticas anteriores
- Al comienzo de todos los ficheros fuente entregados debes incluir un comentario con tu NIF (o equivalente) y tu nombre. Por ejemplo:

```
xxx.cc

// DNI 12345678X GARCIA GARCIA, JUAN MANUEL
...
```

- El cálculo de la nota de la práctica y su relevancia en la nota final de la asignatura se detallan en las diapositivas de presentación de la asignatura (*Tema 0*)

1. Descripción de la práctica

En esta práctica simularemos el clásico juego Tetris[1] en modo texto, con algunas simplificaciones:

- Las piezas no bajan solas hacia abajo si el usuario no lo indica
- Las líneas completadas no se eliminan del tablero

Al principio se creará una pieza aleatoria y se posicionará al principio del tablero (arriba), y se pedirá al usuario una orden, que puede ser:

- mover hacia abajo
- mover a la izquierda
- mover a la derecha
- rotar en el sentido de las agujas del reloj
- rotar en sentido contrario al de las agujas del reloj

Cuando una pieza se intente mover hacia abajo y colisione con otras piezas anteriores, se fijará y ya no se podrá mover más. Las posiciones ocupadas por piezas fijadas se representarán en el tablero con `@`, y las posiciones de la pieza que se está moviendo con # (véase la figura 1). Una vez fijada una pieza, se elegirá otra pieza aleatoria y se repetirá el proceso, hasta que no sea posible colocar en el tablero la nueva pieza, como por ejemplo en la figura 2

[1] https://es.wikipedia.org/wiki/Tetris.

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

	0	1	2	3	4	5	6	7	8	9
0										
1				#						
2			#	#						
3				#						
4										
5										
6										
7	@	@								
8	@	@							@	
9	@	@	@		@	@	@	@	@	@

Figura 1: Forma de una pieza `T` rotada 90º en la posición `(1,2)`

	0	1	2	3	4	5	6	7	8	9
0					@					
1				@	@	@				
2					@					
3				@	@	@				
4				@						
5			@	@						
6		@	@	@	@					
7	@	@								
8	@	@							@	
9	@	@	@		@	@	@	@	@	@

Figura 2: Final de una partida (no se puede colocar una pieza en el centro del tablero en la primera fila)

2. Detalles de implementación

En el *Moodle* de la asignatura se publicarán varios ficheros que necesitarás para la correcta realización de la práctica:

- `prac3.cc`. Fichero que contiene el `main` de la práctica. Se encarga de crear los objetos de las clases implicadas en el problema, y de pedir al usuario las órdenes para mover la pieza, así como la gestión del final del juego. Este fichero no debe incluirse en la entrega final, es solamente una ayuda para probar la práctica, y se puede modificar o usar como base para otras pruebas
- `Shape.h` y `Shape.cc`. Una clase que permite elegir aleatoriamente el tipo de la nueva pieza, y obtener una matriz con su forma, dependiendo del tipo de pieza y su orientación (más adelante se explica con más detalle). Tampoco debe incluirse estos ficheros en la entrega final
- `makefile`. Fichero que permite compilar de manera óptima todos los ficheros fuente de la práctica y generar un único ejecutable
- `autocorrector-prac3.tgz`. Contiene los ficheros del autocorrector para probar la práctica con varias pruebas unitarias que evalúan los métodos por separado

2.1. Compilación

En esta práctica cada una de las clases se implementará en un módulo diferente, de manera que tendremos dos ficheros para cada una de ellas: `Coordinate.cc` y `Coordinate.h` para gestionar las posiciones del tablero, `Piece.cc` y `Piece.h` para gestionar las piezas, `Gameboard.cc` y `Gameboard.h` para manejar el tablero, que se implementará como una matriz bidimensional de punteros a piezas y `Game.cc` y `Game.h` para controlar los movimientos y el juego. Estos ficheros, junto con `prac3.cc`, `Shape.cc` y `Shape.h`, se deben compilar conjuntamente para generar un único ejecutable. Una forma de hacer esto es de la siguiente manera:

Terminal

```
$ g++ Coordinate.cc Piece.cc Gameboard.cc Game.cc Shape.cc prac3.cc -o prac3
```

Esta solución no es óptima, ya que compila de nuevo todo el código fuente cuando puede que solo alguno de los ficheros haya sido modificado. Una forma más eficiente de realizar la compilación de código distribuido en múltiples ficheros fuente es mediante la herramienta `make`. Debes copiar el fichero `makefile`

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

proporcionado en Moodle dentro del directorio donde estén los ficheros fuente e introducir la siguiente orden:

Terminal
```
$ make
```

- Puedes consultar la transparencia 61 en adelante del *Tema 5* si necesitas más información sobre el funcionamiento de `make`

2.2. Excepciones

Algunos de los métodos que vas a crear en esta práctica deben lanzar excepciones, que son un mecanismo especialmente útil en los constructores porque evitan construir objetos con valores incorrectos.

Para lanzar una excepción debes utilizar `throw` seguido del tipo de excepción, que en C++ puede ser cualquier valor (un entero, una cadena, etc), pero en esta práctica debe ser siempre la excepción estándar `invalid_argument` (que nunca se debe capturar):

Terminal
```
if (...)
 throw invalid_argument(mensaje);
```

donde `mensaje` es una cadena (`string` o vector de `char`). Si el argumento inválido es un número, se puede convertir a cadena con la función `to_string`:

Terminal
```
if (...)
 throw invalid_argument(to_string(numero));
```

Para usar la `invalid_argument` debes añadir en tu código `#include <stdexcept>`

3. Clases y métodos

En la figura 3 se muestra un diagrama UML con las clases que hay que implementar, junto con los atributos, métodos y relaciones que tienen lugar en el escenario de esta práctica. **En esta práctica no está permitido añadir ningún atributo o método público a las clases definidas en el diagrama, ni añadir o cambiar argumentos de los métodos. Si necesitas incorporar más métodos y atributos a las clases descritas en el diagrama, puedes hacerlo, pero siempre incluyéndolos en la parte privada de las clases.** Recuerda también que las relaciones de agregación y composición dan lugar a nuevos atributos cuando se traducen del diagrama UML a código. Consulta las transparencias 58 y 59 del Tema 5 si tienes dudas sobre cómo traducir las relaciones de agregación y composición a código. A continuación se describen los métodos de cada clase. Es posible que algunos de estos métodos no sea necesario utilizarlos en el `prac3.cc`, pero se utilizarán en las pruebas unitarias durante la corrección. Se recomienda implementar las clases en el orden en que aparecen en este enunciado.

3.1. Shape

En el Moodle de la asignatura se publicará esta clase (que no debes modificar ni entregar con la práctica), que contiene tres métodos estáticos y los tipos enumerados `PieceType` y `Orientation` (definidos en el fichero `Shape.h`). El tipo enumerado `PieceType` representa los tipos de pieza del juego con la letra

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

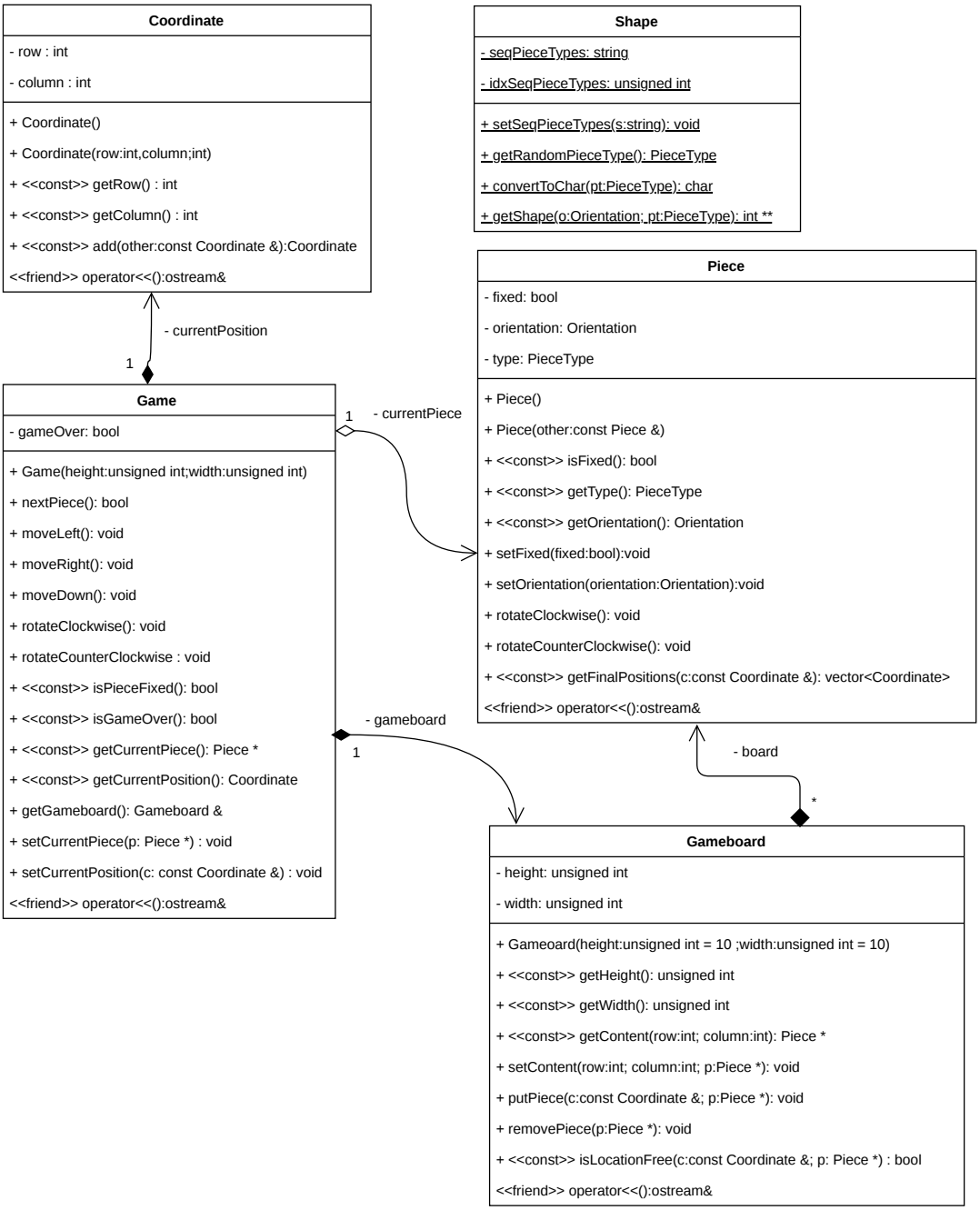


Figura 3: Diagrama UML de las clases de la práctica

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

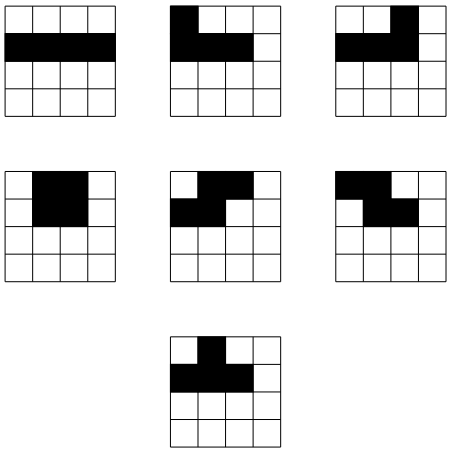


Figura 4: Tipos de pieza: I, J, L, O, S, Z, T (sin rotar, a 0º)

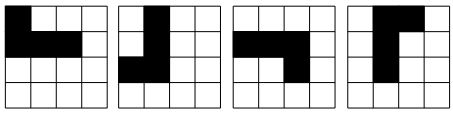


Figura 5: La pieza J con orientación 0º, 90º, 180º y 270º

mayúscula más parecida a la pieza: las letras son `I`, `J`, `L`, `O`, `S`, `Z` y `T`. La figura 4 muestra los diferentes tipos de pieza.

La orientación indica la rotación de la pieza: 0º, 90º, 180º y 270º. Por ejemplo, la figura 5 muestra la pieza J con todas las orientaciones posibles.

Esta clase proporcionará una matriz de 4x4 enteros en la que cada posición ocupada por la pieza se representará con un 1, y las posiciones libres con un 0. Una pieza cualquiera ocupará siempre 4 de las 16 posiciones posibles de la matriz 4x4

La clase `Shape` tiene dos variables de clase, `seqPieceTypes` (una cadena) e `idxSeqPieceTypes` (el índice que recorre la cadena) que se pueden utilizar para establecer las primeras piezas que se usen en el juego; cuando la secuencia se termine, se generarán piezas aleatoriamente.

Los métodos de `Shape` son:

- `PieceType getRandomPieceType()`: devuelve un tipo de pieza aleatorio de entre los 7 posibles. Se debe llamar a este método desde el constructor de `Piece`
- `void setSeqPieceTypes(string s)`: permite almacenar una secuencia inicial de tipos de piezas (la variable de clase `seqPieceTypes`) que el método `getRandomPieceType` consultará antes de generar un tipo de pieza aleatorio; mientras queden tipos de pieza por devolver en la secuencia, el método `getRandomPieceType` devolverá un tipo de la secuencia; cuando se termine la secuencia los tipos de piezas serán aleatorios. Esto permite realizar pruebas con tipos de pieza concretos, llamando a `setSeqPieceTypes` desde el programa principal, antes de comenzar la partida
- `char convertToChar(PieceType pt)`: convierte un tipo de pieza a la letra equivalente
- `int **getShape(Orientation o,PiceceType pt)`: devuelve una matriz 4x4 con las posiciones que va a ocupar una pieza según su orientación y el tipo de pieza. Por ejemplo, para la pieza de la figura 1 (una pieza T rotada 90º) devolvería esta matriz:

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

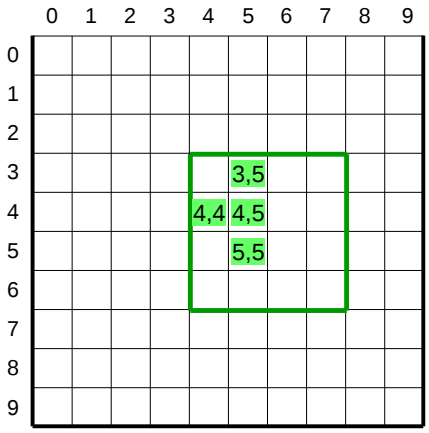
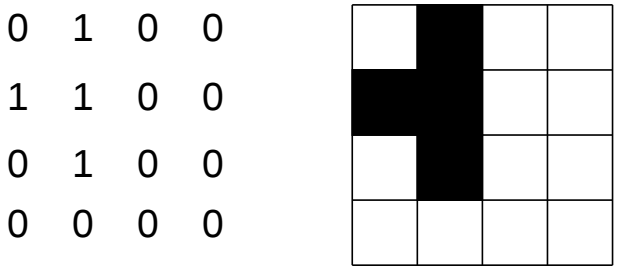


Figura 6: La pieza T con orientación 90º en la posición `[3,4]` ocupa las posiciones `[3,5]`, `[4,4]`, `[4,5]` y `[5,5]`



A partir de esta matriz y la posición de la pieza en el tablero (que se corresponde con la esquina superior izquierda, la posición $(0,0)$ de esta matriz) se debe obtener las posiciones finales que va a ocupar la pieza en el tablero (de esto se encargará el método `getFinalPositions` de la clase `Piece`).

Las posiciones que en la matriz tienen un 1 serán las ocupadas por la pieza; si por ejemplo la posición de la pieza fuera la `[3,4]`, las posiciones ocupadas por la pieza de la figura 6 serían la `[3,5]`, la `[4,4]`, la `[4,5]` y la `[5,5]`

Para la pieza de la figura 5 devolvería una de las siguientes matrices, dependiendo de la orientación

```
1 0 0 0     0 1 0 0     0 0 0 0     0 1 1 0
1 1 1 0     0 1 0 0     1 1 1 0     0 1 0 0
0 0 0 0     1 1 0 0     0 0 1 0     0 1 0 0
0 0 0 0     0 0 0 0     0 0 0 0     0 0 0 0
```

3.2. Coordinate

Esta clase gestiona las coordenadas de las piezas en el tablero, almacenando la fila y la columna. No hay restricciones en los valores que pueden tomar las coordenadas, pueden estar fuera del tablero e incluso pueden ser negativos. Los métodos son:

- `Coordinate()`: constructor que inicializa `row` y `column` a 0
- `Coordinate(int row,int column)`: constructor que inicializa los atributos con los valores pasados como parámetros
- `int getRow() const`: *getter* que devuelve el valor de la fila

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

- `int getColumn() const`: *getter* que devuelve el valor de la columna
- `Coordinate add(const Coordinate &coord) const`: devuelve una coordenada nueva resultado de sumar la coordenada que recibe la llamada (`*this`) con `coord`, sumando las filas y las columnas. Ejemplo:

```
Coordinate a(3,7), b(1,5), c(-5,-3);
cout << a.add(b) << endl;    // [4,12]
cout << a.add(c) << endl;    // [-2,4]
```

- `ostream& operator<<(ostream &os,const Coordinate &c)`: operador salida que muestra la coordenada. Por ejemplo, si la fila es 2 y la columna 7 mostrará `[2,7]` (sin `endl` ni `\n`)

3.3. Piece

Esta clase maneja las piezas del juego. Los métodos son:

- `Piece()`: constructor que inicializa los atributos de la pieza, poniendo el atributo `fixed` (que indica si la pieza se puede mover o no) a `false`, la orientación a `D0` y llamando a `Shape::getRandomPieceType()` para obtener un tipo de pieza aleatorio y almacenarlo en `type`
- `Piece(const Piece &p)`: constructor de copia
- `isFixed, getType, getOrientation`: *getters* básicos
- `setFixed, setOrientation`: *setters* para modificar atributos del objeto
- `void rotateClockwise()`: rotar 90º la pieza en el sentido de las agujas del reloj; por ejemplo, si la orientación es `D0` pasaría a ser `D270`, de ahí a `D180`, luego a `D90` y luego volvería a `D0`
- `void rotateCounterClockwise()`: rotar 90º la pieza en el sentido contrario al de las agujas del reloj; por ejemplo, si la orientación es `D0` pasaría a ser `D90`, luego `D180`, a continuación `D270` y finalmente a `D0`
- `vector<Coordinate> getFinalPositions(const Coordinate &coord) const`: devuelve un vector con las coordenadas que ocuparía la pieza si la esquina superior izquierda fuera `coord`, llamando a `Shape::getShape`. Por ejemplo, en la figura 1 hay una pieza de tipo `T` con la orientación `D90`, el método `getShape` devolvería la matriz de 0s y 1s que aparece en la descripción de ese método, y si `coord` fuera la `[1,2]`, el vector que se devolvería tendría las coordenadas `[1,3]`, `[2,2]`, `[2,3]` y `[3,3]` (el orden de las coordenadas en el vector debe ser el obtenido al recorrer la matriz 4x4 que devuelve `getShape`); si `coord` fuera la `[3,4]` (figura 6) devolvería las coordenadas `[3,5]`, `[4,4]`, `[4,5]` y `[5,5]`
- `ostream& operator<<(ostream &os,const Piece &p)`: operador salida que muestra los datos de la pieza terminados en `\n`, como en este ejemplo:

  ```
  [type=Z, fixed=false, orientation=90]
  ```

  El tipo es la letra mayúscula asociada al tipo de pieza (un valor I, J, L, O, S, Z, T), a continuación se muestra si la pieza está fijada o no, y finalmente se muestra la orientación: `0`, `90`, `180` o `270`

3.4. Gameboard

Clase para gestionar el tablero del juego, que será una matriz rectangular de `height` filas y `width` columnas de punteros a `Piece`; si la posición está vacía contendrá `nullptr`, y si está ocupada por una pieza tendrá un puntero a esa pieza; como cada pieza ocupa 4 posiciones de las 16 de la matriz 4x4 (las que tengan un 1), habrá 4 posiciones del tablero apuntando a la misma pieza. El siguiente fragmento de código crea e inicializa el tablero:

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

```
// El tablero es un atributo privado y se declara así: Piece ***board;

// Para inicializar el tablero hay que hacer lo siguiente
board=new Piece **[height];
for(unsigned int i=0;i<height;i++){
    board[i]=new Piece *[width];
    for(unsigned int j=0;j<width;j++){
        board[i][j]=nullptr; // Se inicializa cada casilla a nullptr
    }

// A partir de aquí se usa board[i][j], que tendrá un puntero a una pieza o nullptr
```

Los métodos son:

- `Gameboard(unsigned int height=10,unsigned int width=10)`: constructor que asigna los valores de los atributos y reserva memoria para la matriz de punteros. Tanto `height` como `width` deben tener un valor mínimo de 10 (que es el valor por defecto si no se indica) y máximo de 20; en caso contrario, se debe lanzar la excepción `invalid_argument` con el mensaje "`wrong board size`"
- `getHeight, getWidth`: *getters* básicos
- `Piece *getContent(int row,int column) const`: devuelve el contenido de la posición del tablero indicada por `row` y `column`, o bien devuelve `nullptr` si la coordenada no está dentro del tablero
- `void setContent(int row,int column,Piece *p)`: guarda `p` en la posición indicada por la coordenada (si está dentro del tablero, si está fuera no hace nada)
- `void putPiece(const Coordinate &c,Piece *p)`: obtiene las coordenadas finales de la pieza `p` en la coordenada `c` del tablero (por ejemplo, en la figura 6 serían `[3,5]`, `[4,4]`, `[4,5]` y `[5,5]`), y almacena la pieza en cada una de esas posiciones (llamando a `setContent`). Se debe suponer que se habrá comprobado antes que esas posiciones pueden ser ocupadas por la pieza sin problemas
- `void removePiece(Piece *p)`: borra del tablero la pieza `p`, cambiando a `nullptr` las posiciones que ocupaba
- `bool isLocationFree(const Coordinate &c,Piece *p) const`: devuelve `true` si todas las posiciones finales de la pieza están dentro del tablero, y no están ocupadas o bien están ocupadas por una pieza no fijada[2]
- `ostream& operator<<(ostream &os,const Gameboard &gb)`: operador de salida que muestra el tablero como en el siguiente ejemplo (similar a la figura 1):

```
|           |
|   #       |
|  ##       |
|   #       |
|           |
|           |
|           |
|@@         |
|@@       @ |
|@@@ @@@@@@|
============
```

  Las posiciones ocupadas por piezas fijadas se representan con `@`, las que ocupa la pieza que está bajando (que no está fija) con `#` y las posiciones libres con un espacio en blanco. En las paredes verticales se pondrá una barra vertical (`|`) y en el final se pone un símbolo de igual (`=`); todas las líneas terminan con `\n`

[2]En cualquier momento del juego solamente hay una pieza sin fijar, la que está bajando, por lo que para poder moverla o girarla debemos considerar las posiciones que ocupa como libres.

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

3.5. Game

Clase para gestionar el juego de Tetris. Almacena un tablero, un puntero a la pieza actual (la que se está moviendo), su posición y un booleano `gameOver` para indicar si la partida ha terminado. Los métodos son:

- `Game(unsigned int height,unsigned int width)`: crea un tablero con las dimensiones pasadas como parámetro, inicializa a `nullptr` la pieza actual, a `false` el atributo `gameOver`, y se deja `currentPosition` con el valor por defecto que asigna el constructor de `Coordinate`
- `bool nextPiece()`: crea una nueva pieza y calcula la posición inicial en la primera fila, en la posición central del tablero. Si `width` vale 10, la posición sería la `[0,3]` puesto que la pieza ocupa cuatro posiciones (virtualmente ocupa una matriz de 4x4 posiciones, aunque sólo 4 posiciones de la matriz estarán ocupadas, depende del tipo de pieza y la orientación); si `width` vale 11 sería también la `[0,3]`, y si vale 20 sería la posición `[0,8]`. Si las posiciones que va a ocupar la pieza están libres, se pondrá la pieza en el tablero, se actualizarán los atributos del juego y se devolverá `true`. Si no es posible colocar la nueva pieza el juego habrá terminado, se pondrá `gameOver` a `true` y este método devolverá `false`
- `moveLeft, moveRight, moveDown`: mueven una posición la pieza. Si la partida no ha terminado y la pieza actual no está fijada, se calculará la nueva posición de la pieza y se comprobará si es posible moverla, en cuyo caso se moverá en el tablero y se actualizará la posición de la pieza. En el caso de `moveDown`, si la pieza no se puede mover se quedará fijada y ya no será posible moverla más
- `rotateClockwise, rotateCounterClockwise`: si la partida no ha terminado y es posible mover la pieza actual, se calculará si sería posible rotar en el sentido de las agujas del reloj (`rotateClockwise`) o en sentido contrario al de las agujas del reloj `rotateCounterClockwise`) la pieza, y se rotará en caso afirmativo
- `isGameOver`: indica si la partida ha terminado
- `isPieceFixed`: indica si la pieza actual ha sido fijada o no
- `ostream& operator<<(ostream &os,const Gameboard &gb)`: operador de salida que muestra solamente el tablero del juego (usando el operador correspondiente de `Gameboard`)
- `getCurrentPiece, getCurrentPosition, setCurrentPiece, setCurrentPosition`: *getters* y *setters* para facilitar las pruebas unitarias.
- `getGameboard`: método que devuelve el tablero por referencia, para que en las pruebas unitarias sea posible modificarlo fuera de la clase. En condiciones normales, no se debe devolver por referencia ningún atributo, y tampoco se deben implementar `setters` para atributos críticos (como `currentPosition` y `currentPiece`), pero en esta práctica es necesario incluir todos estos métodos para facilitar las pruebas unitarias que se usarán para corregir.

4. Programa principal

El programa principal está en el fichero `prac3.cc` publicado en el Moodle de la asignatura, y no se debe entregar con la práctica.

El programa crea un objeto de la clase `Game`, crea la primera pieza y va pidiendo al usuario órdenes para mover la pieza; si la pieza resulta fijada se creará una nueva pieza y se seguirá pidiendo órdenes, hasta que la partida termine. A diferencia del juego de Tetris original, las líneas completadas en el tablero no se eliminan, y la pieza actual no cae automáticamente, sólo se mueve hacia abajo si el jugador lo indica.

Las órdenes que admite el programa son "`l`" para mover a la izquierda (left), "`r`" para la derecha (right), "`d`" para rotar en el sentido de las agujas del reloj y "`u`" para rotar en sentido contrario; si se introduce una cadena vacía se moverá hacia abajo.

```
#include <iostream>
#include <ctype.h>
```

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

```
using namespace std;

#include "Game.h"

int main()
{
  srand(777); // Inicializar generador de números aleatorios
  Shape::setSeqPieceTypes("OSLOJIZOT"); // Secuencia inicial de piezas, luego aleatorias

  string linea;
  Game game(12,10);

  game.nextPiece();
  do
  {
    cout << game;
    cout << "Orden: "; getline(cin,linea);

    if (linea == "")
    {
      game.moveDown();
      // Chequear fixed + nextPiece + gameOver
      if (game.isPieceFixed())
      {
        game.nextPiece();
        if (game.isGameOver()) {
          cout << game << endl
               << "GAME OVER!!" << endl;
          break;
        }
      }
    }
    else if (linea == "L" || linea == "l") game.moveLeft();
    else if (linea == "R" || linea == "r") game.moveRight();
    else if (linea == "U" || linea == "u") game.rotateCounterClockwise();
    else if (linea == "D" || linea == "d") game.rotateClockwise();

  } while (linea != "q");
}
```

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional